Programmazione strutturata
(Dijkstra, 1969)

La programmazione strutturata nasce come proposta per regolamentare e standardizzare le metodologie di programmazione.

Obiettivo:

rendere piu' facile la lettura dei programmi (e quindi la loro modifica e manutenzione).

› La parte istruzioni (o parte esecutiva) di un programma viene vista come un ***comando complesso o strutturato*** ottenuto componendo ***comandi elementari*** (assegnamento, chiamata di procedura), mediante alcune regole di composizione (***strutture di controllo***).

Strutture di controllo:

- **concatenazione** (o composizione);
- **alternativa** (o istruzione condizionale) che ramifica il flusso di controllo in base al valore di un'espressione (logica o relazionale);
- **ripetizione** (o iterazione) che provoca l'esecuzione ripetuta di un'istruzione (eventualmente composta) finche` un'espressione booleana rimane vera.

Strutture di Controllo

Caratteristiche:

- Hanno un solo punto di ingresso ed un solo punto di uscita.

- Possono essere interpretati come una singola azione in una computazione sequenziale.

- I programmi sono piu` leggibili.

Le strutture di concatenazione, iterazione e condizionali costituiscono un insieme completo in grado di esprimere tutte le funzioni calcolabili.

☞ L'uso di queste sole strutture di controllo non limita il potere espressivo.

Istruzioni semplici

In C, qualsiasi espressione seguita da un punto e virgola è un'istruzione

```
x= 0;          /* istruzione di assegnamento */
y = 1;         /* istruzione di assegnamento */
x = x+y;       /* istruzione di assegnamento*/


k++;           /* istruzione di postincremento */

printf(...);   /* istruzione di uscita
                  (chiamata di funzione) */

;              /* istruzione nulla */
```

Istruzioni strutturate:

- **istruzione composta:** (blocco)

```
{ <sequenza-istruzioni> }
```

- **istruzione condizionale:** (alternativa)

```
if (<espressione> )
  <istruzione>;
else
  <istruzione>;
```

- **istruzione condizionale:** (alternativa multiple)

```
switch (<espressione intera o carattere>)
{
  case <costante1>: <blocco1>; [break;]
  case <costante2>: <blocco2>; [break;]
  ...
  case <costanteN>: <bloccoN>; [break;]
  [default: <bloccoDiDefault>;]
}
```

- **istruzione di iterazione:** (ciclo)

```
while (<espressione>)
  <istruzione>;
```

- **istruzione di iterazione:** (ciclo)

```
do
        <istruzione>;
while (<espressione>);
```

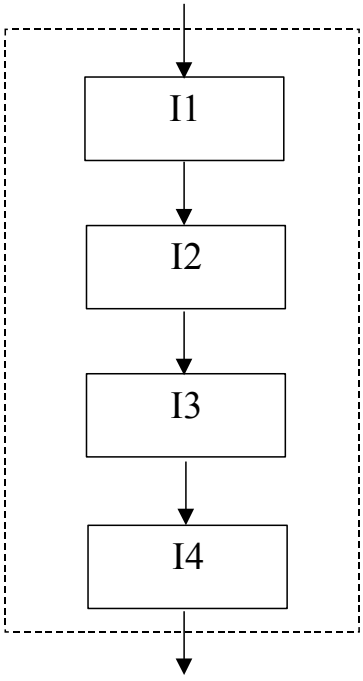
- **istruzione ripetivitive** (ciclo)

```
for (<espressione1>;<espr2>;<espr3>;)
  <istruzione>;
```

Istruzione composta

```
{
   <Dichiarazioni e Definizioni>
   <Sequenza di Istruzioni>
}
```

Sintatticamente equivalente a una singola istruzione.



E` possibile definire variabili che hanno visibilità e tempo di vita limitato al blocco stesso

La parte istruzioni del main e` essa stessa un'istruzione composta.

Esempio: istruzione composta

```
/*programma che letti due numeri a
  terminale ne stampa la somma*/
#include <stdio.h>
main()
{
int X,Y;

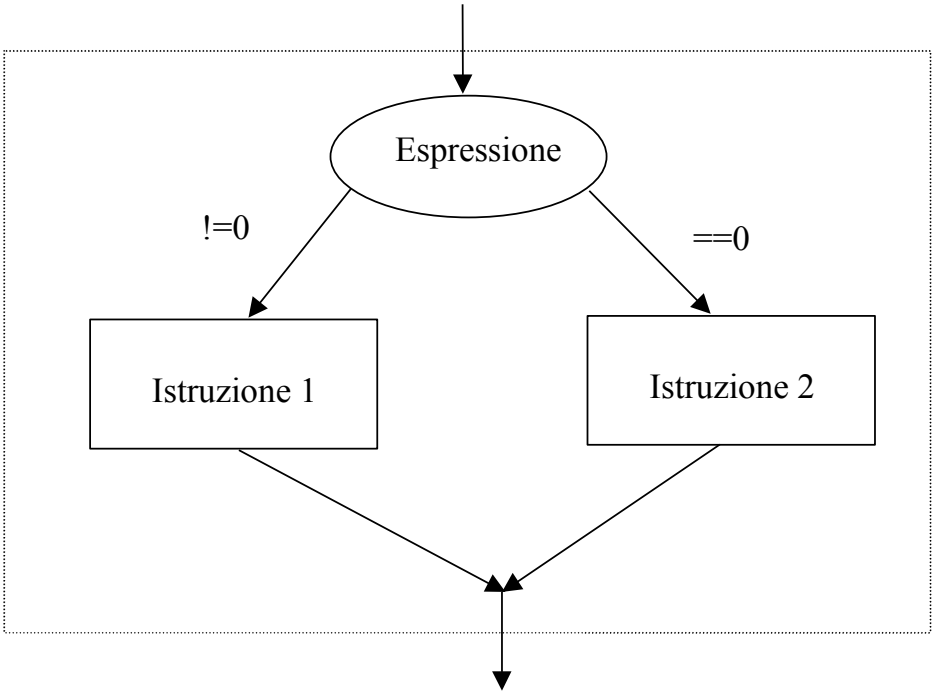
    scanf("%d%d",&X,&Y);
    printf("%d",X+Y);
}
```

Istruzioni di alternativa

Istruzione if:

if (<espressione>)
    <istruzione1>
else
    <istruzione2>;

se il risultato di espressione è *vero* (diverso da zero) viene eseguita l'istruzione 1, altrimenti viene eseguita l'istruzione 2.



La parte **else** è opzionale.

```
if (Eta>=100)
    printf("%s", "centenario");
else
    printf("%d", Eta);
```

Istruzione if:

if (<espressione>)
    <istruzione1>
else
    <istruzione2>;

- <istruzione1> ed <istruzione2> possono essere di tipo qualunque, semplice o strutturato, eventualmente anche un'istruzione composta:

```
if ((Eta >=6) && (Eta <=14))
   printf("%s","scolare");
else {
      printf("%s","Non scolare");
      printf("%d",Eta);
   };
```

Esempio:

Programma che legge due numeri e determina qual e` il maggiore.

```
/* determina il maggiore tra due numeri
file c2.c */

#include <stdio.h>
main()
{
  int primo,secondo;

  scanf("%d%d",&primo,&secondo);
  if (primo>secondo)
       printf("%d",primo);
  else printf("%d",secondo);
}
```

Esempio: if

```
/* Programma che calcola le radici di
un'equazione di secondo grado*/

#include <stdio.h>
#include <math.h> /*lib. matematica*/
main()
{
  int a,b,c;
  float  d,x1,x2;

scanf("%d%d%d",&a,&b,&c);
if ((b*b) < (4*a*c))
 printf("%s","radici complesse");
else
  {
   printf("%d%d%d",a,b,c);
   d=sqrt(b*b-4*a*c);
   x1=(-b+d)/(2*a);
   x2=(-b-d)/(2*a);
   printf("%f%f",x1,x2);
  };
}
```

✏ **Esercizio:**
Risolvere un sistema lineare di due equazioni in due incognite

$$a_1x + b_1y = c_1$$
$$a_2x + b_2y = c_2$$

$x = (c_1b_2-c_2b_1) / (a_1b_2- a_2b_1) = XN / D$
$y = (a_1c_2- a_2c_1) / (a_1b_2- a_2b_1) = YN / D$

Soluzione:

```
#include <stdio.h>
main()
{
  int    A1,B1,C1,A2,B2,C2,XN,YN,D;
  float  X,Y;

  scanf("%d%d%d%\n",&A1,&B1,&C1);
  scanf("%d%d%d%\n",&A2,&B2,&C2);
  XN = (C1*B2 - C2*B1);
  D =  (A1*B2 - A2*B1);
  YN = (A1*C2 - A2*C1);
  if (D == 0)
     {if (XN == 0)
        printf("sist. indeterm.\n");
      else
        printf("Nessuna soluz.\n");
     }
  else
     {X= XN/D;
      Y= YN/D;
      printf("%f%f\n",X,Y);
     }
}
```

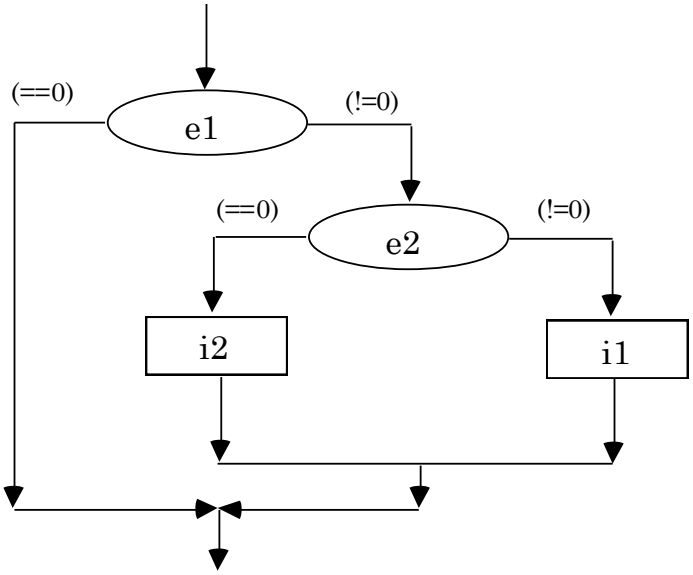
Istruzione if:

if (<espressione>)
    <istruzione1>
else
    <istruzione2>;

- <istruzione1> ed <istruzione2> possono essere ancora istruzioni if.

☞ Si possono avere piu` **istruzioni if annidate**:

if (<e1>)
    if (<e2>)
        <i1>;
    else
        <i2>;



L'**else** si riferisce sempre all'**if** più "interno", se non specificato altrimenti.

```
if (n > 0)
  if (a > b)
      n = a;      /*n > 0 && a > b   */
  else   n = b; /*  n > 0 && a <= b */
```

☞ Se si vuole riferire l'**else** all'**if** piu` esterno, si usano le graffe:

```
if (n > 0)
  { if (a > b) n = a; }/*n>0 && a>b */
else
  n = b;          /*  n <= 0   */
```

If annidati

✏ **Esercizio:**
Dati tre numeri interi positivi che rappresentano i tre lati di un triangolo, si stampi il tipo del triangolo (equilatero, isoscele o scaleno).

Prima specifica:

```
main()
{
/*dichiarazione dati */

    /* leggi le lunghezze dei lati */
    /*se triangolo, stampane il tipo */
}
```

Codifica:

Come dati occorrono tre variabili intere per rappresentare le lunghezze dei segmenti.

```
/*dichiarazione dati */
unsigned int  primo,secondo,terzo;

  /* leggi le lunghezze dei segmenti */
   scanf("%d%d%d",&primo,&secondo,&terzo);

  /*se triangolo, stampane il tipo */
if (primo + secondo>terzo)
  if (primo==secondo)/*det.il tipo */
     {if (secondo==terzo)
         printf("equilatero");
      else
         printf("isoscele");}
  else
     {if (secondo==terzo)
         printf("isoscele");
      else
         {if (primo==terzo)
             printf("isoscele");
          else
             printf("scaleno");}
     }
```

Indentazione:

l'"indent" delle linee del testo del programma rispetta l'annidamento delle varie istruzioni › si aumenta la leggibilita` del programma (modifica piu` facile).

Esempio: if annidati

Stampa di una variabile di tipo enumerato.

```
typedef enum{cuori,picche,quadri,
             fiori}Semi;
seme  Semi;

if (seme==cuori) printf("%s","cuori");
else
 if (seme==picche)
     printf("%s","picche");
 else
     if (seme==quadri)
     printf("%s","quadri");
     else
        if (seme==fiori)
           printf("%s","fiori");
```

Istruzione di Alternativa Multipla: Switch

Switch:

```
switch (<espressione intera o carattere>)
{
 case <costante1>: <blocco1>; [break;]
 case <costante2>: <blocco2>; [break;]
 ...
 case <costanteN>: <bloccoN>; [break;]
 [default: <bloccoDiDefault>;]
}
```

- L'espressione e` detta **selettore**. Deve avere un valore di tipo intero o carattere.

- Ciascuna costante di una etichetta **case** deve essere dello stesso tipo del selettore.

- Un valore puo` comparire al piu` in un'etichetta.

Istruzione Switch

Significato:

Se l'espressione ha un valore che corrisponde ad una costante (per esempio <costante1>), si esegue il <blocco1> e tutti i blocchi dei rami successivi.

- ☞ Le alternative non sono mutuamente esclusive: possibilità di eseguire in sequenza più blocchi contigui di istruzioni.

- ☞ Necessità di **forzare l'uscita** mediante l'istruzione **break** (che provoca l'uscita forzata dallo switch).

Ramo di default:

E` possibile specificare un'etichetta **default**: essa viene eseguita per qualunque valore ritornato da selettore.
In pratica, consente di utilizzare una “alternativa” che viene eseguita se il valore dell'espressione non corrisponde ad alcuna etichetta.

Esempio:

```
#define   GENNAIO 1
#define   FEBBRAIO   2
#define   MARZO   3
...
#define   NOVEMBRE   11
#define   DICEMBRE   12
...
int mese;
...
switch (mese)
{
case GENNAIO:  giorni = 31;  break;
case FEBBRAIO:
  if (bisestile) giorni = 29;
  else giorni = 28;
  break;
case MARZO:  giorni = 31;  break;
...
case DICEMBRE: giorni = 31;
}
```

2a soluzione:

```
switch (mese)
{
case FEBBRAIO:
  if (bisestile) giorni = 29;
  else giorni = 28;
  break;
case APRILE:  giorni = 30;  break;
case GIUGNO:  giorni = 30;  break;
case SETTEMBRE:  giorni = 30;  break;
case NOVEMBRE:  giorni = 30;  break;
default:  giorni = 31;
}
```

3a soluzione:

```
switch (mese)
{
case FEBBRAIO:
  if (bisestile) giorni = 29;
  else giorni = 28;
  break;
case APRILE:
case GIUGNO:
case SETTEMBRE:
case NOVEMBRE:  giorni = 30;  break;
default:  giorni = 31;
}
```

Esempio: Una calcolatrice

Si vuole realizzare un programma che emuli il comportamento di una calcolatrice in grado di effettuare le operazioni aritmetiche su due operandi A e B.
Si supponga che l'utente immetta l'operazione nel formato:

A *op* B =

dove *op* puo` valere +, -, *, /, %, :.

```
#include <stdio.h>

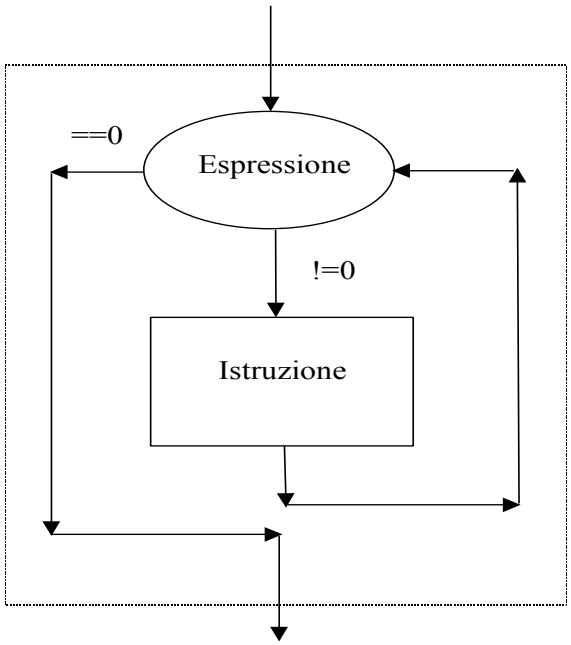
main()
{
int A, B;
char op;

printf("Digita l'operazione che vuoi effettuare
  [formato da usare A op B=] :\n");
scanf("%d%c%d =", &A, &op, &B);
switch (op)
  {
  case '+': printf("%d\n", A+B);break;
  case '-': printf("%d\n", A-B);break;
  case '*': printf("%d\n", A*B);break;
  case '%': printf("%d\n", A%B);break;
  case '/': printf("%d\n", A/B);break;
  case ':': printf("%f\n",(float)A/(float)B);break; /*
divisione non intera*/
  default: printf("\n operazione non prevista\n");
  }
}
```

Istruzioni di iterazione

Istruzione while:

```
while (<espressione>)
        <istruzione>;
```



L'epressione (condizione di ripetizione) viene valutata all'**inizio di ogni ciclo**.

- Il ciclo viene eseguito finche` il valore dell'espressione rimane diverso da zero (*vero* logico).
- Si esce dal ciclo quando l'espressione restituisce un valore =0 (*falso* logico).
- Se inizialmente ha valore zero, il corpo del ciclo non viene **mai** eseguito.

Esempi:

```
somma=0;
j = 1;
while (j <= 10)
  {somma = somma + j;
   j++;
  }


while (Errore==0) && (Car<>‘ ‘)
  {
   scanf("%c",&Car);
   ...
}
```

✏ **Esercizio:**

Scrivere un programma che calcoli la media degli N voti riportati da uno studente.

```
/* Media di n voti */
#include <stdio.h>
main()
{
  int    sum,voto,N,i;
    float  media;

    printf("Quanti sono i voti ?");
    scanf("%d",&N);
   sum=0;
/* ripeti ...*/
    printf("Dammi un voto:");
   scanf("%d",&voto);
   sum=sum+voto;
/* ... per N volte */
   media=sum/N;
   printf("Risultato: %f",media);
}
```

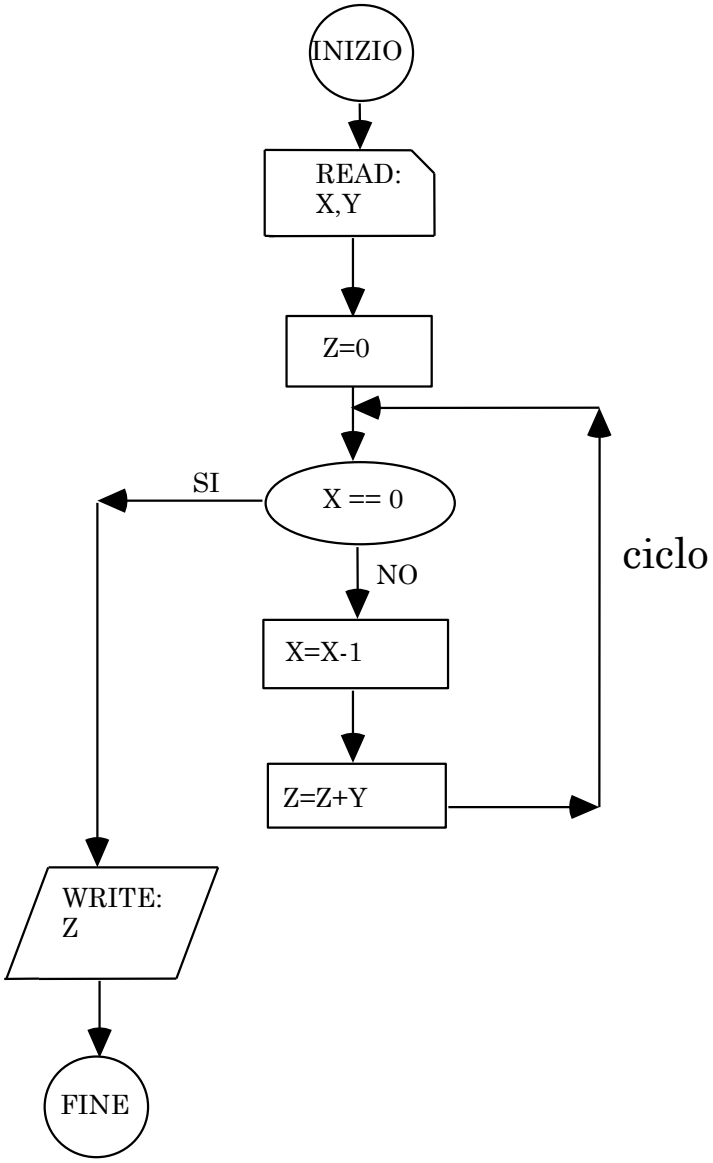
Esercizio (continua):

```
/* Media di n voti*/
#include <stdio.h>
main()
{
  int    sum,voto,N,i;
  float  media;

  printf(“Quanti sono i voti ?”);
  scanf("%d",&N);
  sum=0;
  i=1;
  while (i <= N)
    { /* corpo ciclo while */
    printf(“Dammi il voto n.%d:”,sum);
    scanf("%d",&voto);
    sum=sum+voto;
    i=i+1;
    }
  media=sum/N;
  printf("Risultato: %f",media);
}
```

✏ Esercizio:

Programma che calcola il prodotto X*Y come sequenza di somme (si suppone Y>0, X >=0).



Codifica:

```
/* moltiplicazione come sequenza di
   somme */
#include <stdio.h>
main()
{
  int  X,Y,Z;

  printf(“Dammi i fattori:”);
  scanf("%d%d",&X,&Y);
  Z=0;
  while (X!=0)
      {  /* corpo ciclo while */
           Z=Z+Y;
           X=X-1;
        }
  printf("%d",Z);
}
```

Cicli Innestati

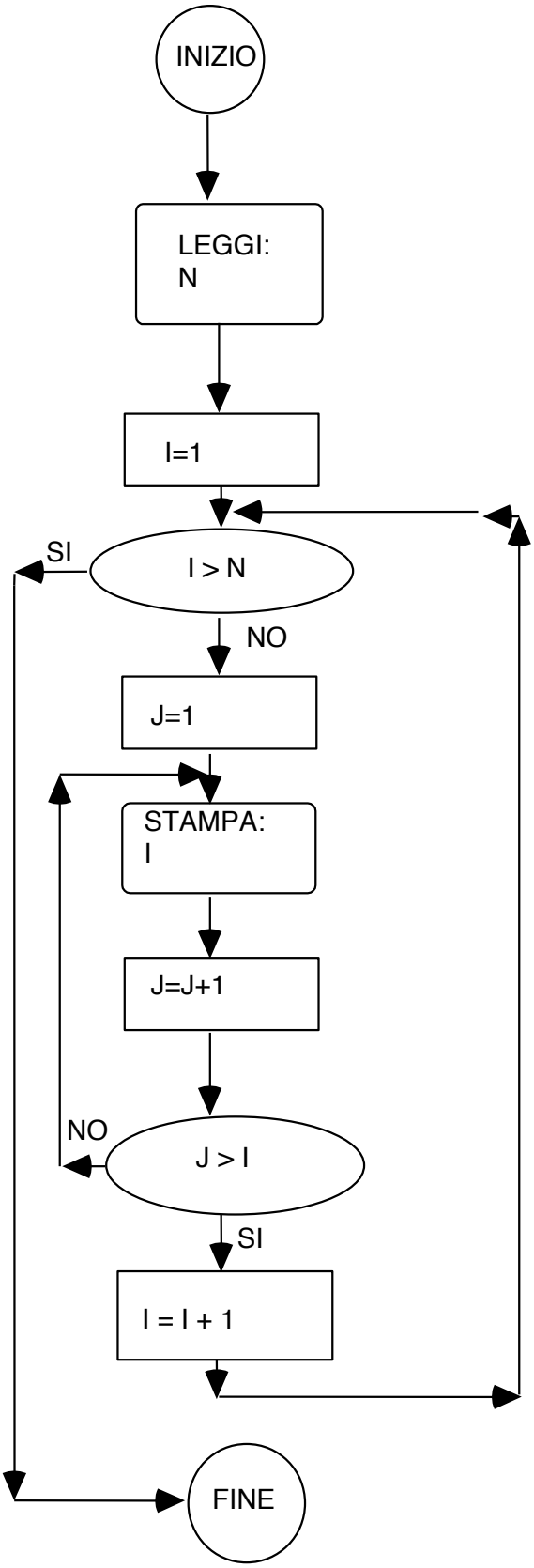
```
while (<espressione>)
        <istruzione>;
```

- <istruzione> puo` essere a sua volta una istruzione while ❯ cicli **innestati**

✏ **Esercizio:**

Si legga un numero naturale N. Stampare una volta il numero 1, due volte il numero 2,..., N volte il numero N.

- Dominio di ingresso (0,1,2,...,N)
- Dominio di uscita (), (1), (1,2,2),(1,2,2,3,3,3)...).


INIZIO
LEGGI:
N
I=1
SI
I > N
NO
J=1
STAMPA:
I
J=J+1
NO
J > I
SI
I = I + 1
FINE

Codifica:

```
#include <stdio.h>
main()
{
  int    N,I,J;

  printf(“dammi N:”);
  scanf("%d",&N);
  I=1;
  while (I<=N)
    {  /* corpo ciclo esterno */
       printf(“Prossimo valore:”);
       printf("%d",I);
       J=1;
       while (J<I)
          {  /* corpo ciclo interno */
             printf("%d",I);
             J=J+1;
          }
       I=I+1;
    }
}
```

✏ **Esercizio:**
Stabilire se un numero naturale N e` primo.
(Un numero naturale N e` primo, se non e` divisibile per alcun numero intero minore di esso.)

Nota: non e` necessario eseguire tutte le divisioni di N per 2, 3, ..., (n-1), ma basta eseguire le divisioni per i naturali minori o uguali alla radice quadrata di N.

```
/* Numero primo */
#include <stdio.h>
#include <math.h>
main()
{
  typedef enum {false,true} boolean;
  int     N, I;
  float   N1;
  boolean primo;

  scanf("%d",&N);
  N1=sqrt(N);
  I=2;primo=true;
  while ((I<=N1) && (primo==true))
    {if (((N / I) * I) == N)
         {primo=false;}
     else  I=I+1; }
  if (primo==true)
      printf("numero primo");
  else printf("numero non primo");
}
```

✏ **Esercizio:**
Calcolo del fattoriale di un numero intero non negativo N:

- fattoriale(0) = 1
- fattoriale(N) = N * (N-1)*...*1=fattoriale(N-1)*N

```
/* Calcolo del fattoriale */
#include <stdio.h>
#include <math.h>
main()
{
  int     N, F, I;

  printf("Dammi N:");
  scanf("%d",&N);
  F=1;
  I=2;
  while (I <= N)
        { F=F*I;
          I=I+1; }
  printf("%s%d","Fattoriale: ",F);
}
```

Esempio:
Divisione tra numeri interi positivi attraverso somma e sottrazione.

```
/* divisione tra interi positivi */
#include <stdio.h>
main()
{
  unsigned int  X,Y,Quoz,Resto;

  scanf("%d%d",&X,&Y);
  Resto=X;
  Quoz=0;
  while (Resto >= Y)
        { Quoz=Quoz+1;
          Resto=Resto-Y; }
  printf("%d%s%d%s%d%s%d",
      X," diviso ", Y, " = ", Quoz,
      " resto ", Resto);
}
```

Istruzioni di iterazione

while (espressione)
istruzione;

- Nell'istruzione **while**, la condizione di ripetizione viene verificata **all'inizio di ogni ciclo**

```
somma=0; j=1;
while (j <= n)
  { somma = somma + j;  j++; }
```

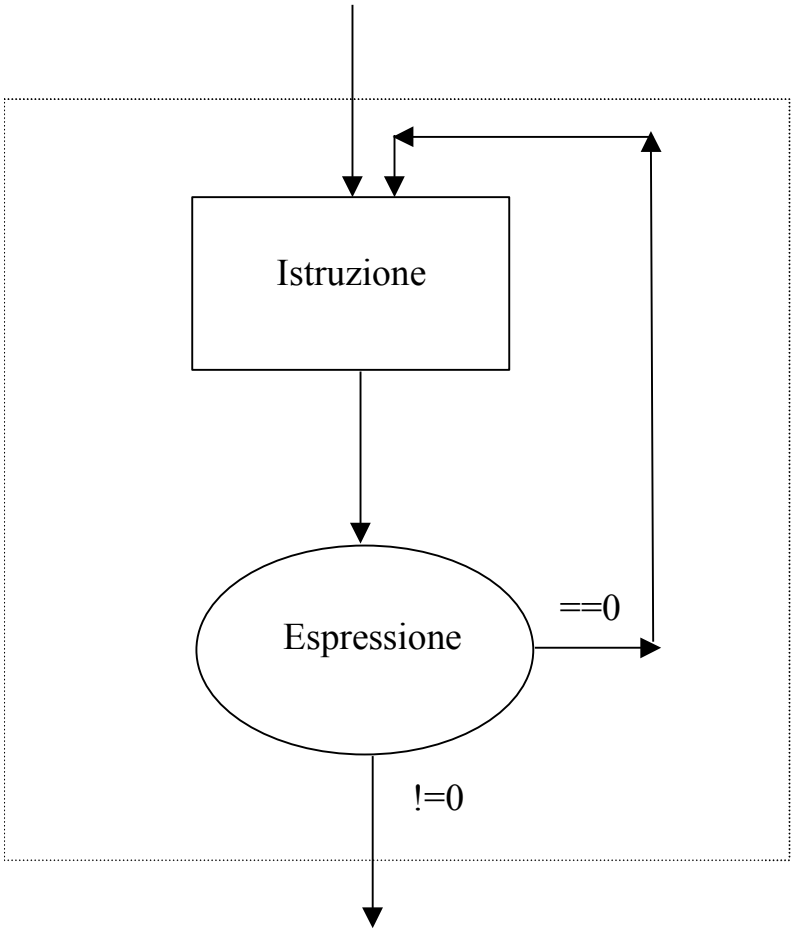
Istruzione do:

do
<istruzione>
while **(<espressione>);**

- La condizione di ripetizione viene verificata **alla fine di ogni ciclo**

```
/* In questo caso: n >0  */
somma = 0; j = 1;
do
  { somma = somma + j;  j++; }
while (j <= n);
```

Istruzione do:



Esempio:

```
do
  scanf("%c",&Car)
while (Car !=' ') /* salta spazi
                     bianchi */
```

Prima dell'esecuzione del ciclo, il valore di Car e` indeterminato.

Con il while:

```
Car=' ';
while (Car==' ')  scanf("%c", &Car);
```

Istruzioni Ripetitive

Istruzione for:

for(<espressione1>; <espressione2>; <espressione3>)
** <istruzione>;**

- <espressione1> e` l'espressione di inizializzazione: viene eseguita una volta sola, prima di entrare nel ciclo
- <espressione2> rappresenta la condizione di permanenza nel ciclo (valutata all'inizio di ogni iterazione)
- <espressione3> e` l'espressione di passaggio al ciclo successivo (valutata alla fine di ogni iterazione).

for è equivalente a:

```
<espressione1>;          /* Inizializzazione  */
while (<espressione2>)/* Condizione di
                                    Ripetizione  */
{
      <istruzione>;
      <espressione3>;  /* InizioNuovoCiclo  */
}
```

Esempio while/for:

```
somma = 0;  /* while */
j = 1;
while (j <= n)
  {
   somma=somma+j;
    j++;
  }
```

con il for:

```
somma = 0; /* for */
for(j=1;j<=n;j++)
  somma = somma + j;
```

For:

```
for(<espressione1>; <espressione2>;  <espressione3>)
   <istruzione>;
```

- Ognuna delle espressioni **può essere omessa** (il punto e virgola deve rimanere)
- Se manca espressione2, si ha un ciclo infinito.

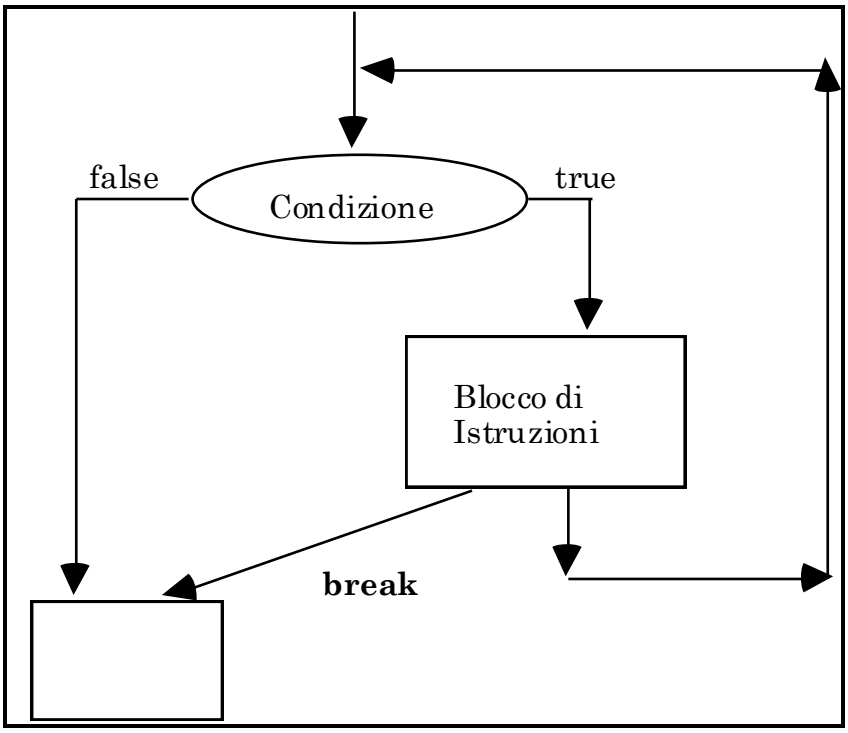
Cosa eseguono i seguenti **for** ?

```
for (i = 1;  i <= n;  i++)  printf("%d ",i);
for (;;)  { ... }
```

Istruzioni per il trasferimento del controllo:

Istruzione break:

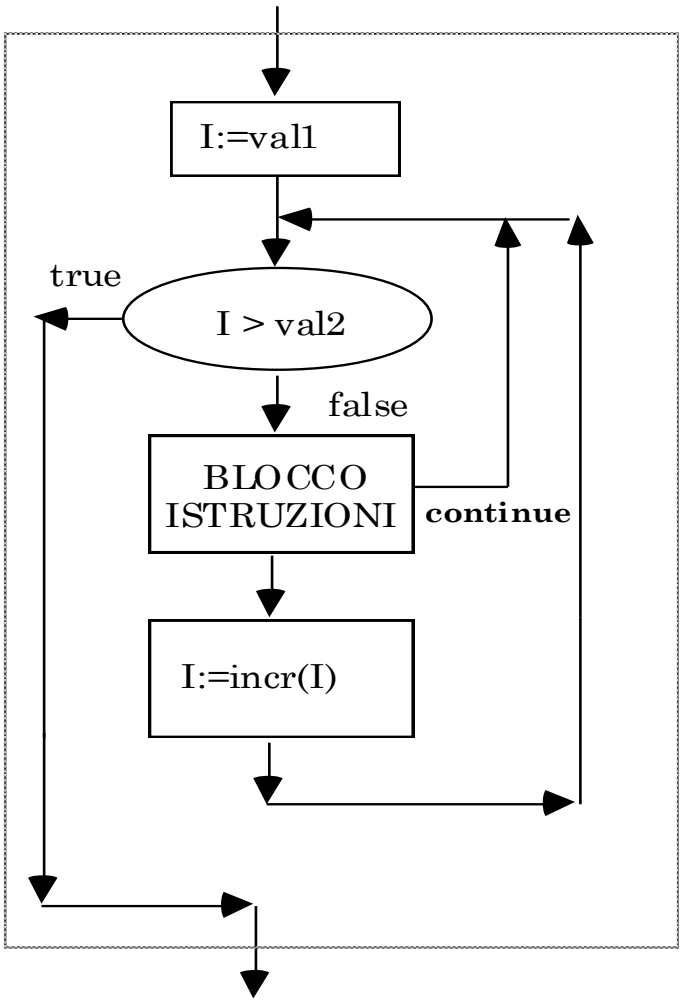
L'istruzione **break** provoca l'uscita immediata dal **ciclo** (o da un'istruzione **switch**) in cui è racchiusa



Istruzione continue:

L'istruzione **continue** provoca l'inizio della successiva iterazione del **ciclo** in cui è racchiusa (non si applica a **switch**).

```
for (i = val1; i <= val2; i++)
{
  ... ;
  if ... continue;
  else ...
}
```



Esempio continue:

```
for (i = N; i > 0; i--)
{
  /* Salta alla prossima ripetizione
 se N è multiplo di i */
  if (N % i) continue;
  /* Esci dal ciclo se i vale 55 */
  if (i == 55) break;
  ...
}
```

Esercizi:

Leggere una sequenza di numeri interi a terminale e calcolarne la somma, nei seguenti casi:

1) La lunghezza della sequenza non e` nota, ma l'ultimo valore e` seguito da un intero negativo (while);

2) Come il caso 1, ma sicuramente la sequenza ha lunghezza maggiore o uguale ad uno (do);

3) La sequenza di numeri da sommare e` preceduta da un numero intero che rappresenta la lunghezza della sequenza (for);

4) La sequenza contiene un intero per ogni linea ed il termine della sequenza di ingresso e` segnalato con il carattere di fine file (Ctrl-Z) (test eof).

Espressioni (assegnamento forma abbreviata)

v ***op***= e è **quasi** equivalente a v = v ***op*** (e)

dove ***op*** è uno dei seguenti operatori:

\+ – * / % >> << & ^ |

```
k += j          /*  equivale a k = k + j  */
k *= a + b      /*  equivale a k = k * (a + b)  */
```

Le due forme sono **quasi equivalenti** perchè in:

v *op*= e

v viene valutato una sola volta, mentre in:

v = v *op* (e)

v viene valutato due volte.

Se la valutazione di v non genera **effetti collaterali** (side effects) le due forme sono del tutto equivalenti,
in caso contrario le due forme non sono equivalenti